🧠 Capa 2: Gestión de Estado Global (Pinia)

En una aplicación pequeña, usar Props y Emits es suficiente. Pero, ¿qué pasa si el volumen del reproductor debe conocerse en 10 páginas distintas? Pasar el volumen por Props a través de todos los componentes sería una pesadilla (esto se llama *Prop Drilling*). Aquí es donde usamos **Pinia**.

---

1. ¿Qué es un Store?

Un **Store** es como un "almacén central" de datos al que cualquier componente puede entrar y sacar (o meter) información sin importar dónde esté en la aplicación.

Las 3 partes de un Store:

1. **State (Variables):** Los datos brutos (ej: `isPlaying`, `currentTrack`).
2. **Getters (Calculados):** Variables que dependen de otras (ej: `hasNext` que mira si queda música en la lista). Son como las `computed` de un componente.
3. **Actions (Funciones):** Donde ocurre la lógica. Pueden ser síncronas o asíncronas (peticiones al servidor).

---

2. Ejemplo Real: `playerStore.ts`

Imagina el reproductor. Necesitamos que siga sonando aunque cambies de página.

```
// app/stores/player.ts
export const usePlayerStore = defineStore('player', () => {
  // 1. STATE
  const currentTrack = ref(null)
  const isPlaying = ref(false)

  // 2. ACTIONS
  function setTrack(track) {
    currentTrack.value = track
    isPlaying.value = true
  }

  function togglePlay() {
    isPlaying.value = !isPlaying.value
  }

  return { currentTrack, isPlaying, setTrack, togglePlay }
})
```

---

3. ¿Cómo se usa en un Componente?

Cualquier componente puede conectarse al almacén:

```
<script setup>
const playerStore = usePlayerStore() // Conexión al almacén
</script>
```

```
<template>
  <div>
    <p>Sonando: {{ playerStore.currentTrack?.title }}</p>
    <button @click="playerStore.togglePlay()">
      {{ playerStore.isPlaying ? 'Pausar' : 'Reproducir' }}
    </button>
  </div>
</template>
```

---

4. Persistencia (Plugin: PersistedState)

Normalmente, si refrescas la página (F5), los datos de JavaScript se borran. Para evitar que el usuario pierda su volumen o su última canción, usamos un plugin que guarda los datos en el `localStorage` del navegador automáticamente.

```
{
  persist: {
    pick: ['volume', 'currentTrack'], // Solo guardamos lo importante
  }
}
```

🚀 Resumen para Estudiantes:

- **Usa Props/Emits** si el dato solo importa entre un padre y su hijo directo.
- **Usa Pinia** si el dato lo necesitan componentes que no tienen relación directa o si el dato debe sobrevivir a la navegación entre páginas.

💡 Regla de Oro:

Nunca modifiques el estado de Pinia directamente desde un componente si la lógica es compleja. Es mejor crear una **Action** en el Store. Esto mantiene tu código ordenado y fácil de depurar.